PsyClaw

Offline-First, RAG-Enforced Soul Agent

Architecture Reference Document | v1.3.0 | May 2026

PsyClaw is a production-grade local AI agent built on three invariants: retrieval always runs first (RAG-first), the LangGraph topology is the security policy (topology = enforcement), and identity is governed through a versioned, crash-safe soul system (identity != memory != topology). Every query path terminates through audit logging with SHA-256 hashing and PII redaction. Version 1.3.0 adds an in-memory per-IP rate limiter (60/min sliding window), expands the prompt-injection sanitizer to 13+ OWASP-aligned banned patterns, and hardens the soul layer with SHA-256 drift detection, atomic evolution writes, and automatic TTL pruning on init. All five security invariants are preserved and structurally strengthened.

v1.3.0 UPDATE HIGHLIGHTS

- Rate limiting fully implemented (in-memory 60/min per-IP sliding window, no slowapi dep)
- Expanded sanitizer: 13 OWASP-aligned banned patterns (was 8)
- Soul hardening: SHA-256 drift detection + auto-recovery, atomic evolution writes, TTL prune on init
- Enhanced test coverage: rate-limit, personality drift, mocked endpoints
- All 5 security invariants preserved and structurally strengthened in code

Property	Value
Version	1.3.0 (rate limit + soul hardening + 13 patterns + drift detection)
Runtime	FastAPI + LangGraph + LM Studio (local GGUF)
Retrieval	ChromaDB (vector) + BM25 (keyword) + RRF fusion
Embeddings	sentence-transformers / all-MiniLM-L6-v2 (CPU, 384-dim)
Soul Governance	SQLite + Markdown, versioned, atomic writes, drift-detected, 365-day TTL
Rate Limiting	In-memory sliding window, 60 req/min per IP, HTTP 429 on exceed
Sanitizer	13 OWASP-aligned banned patterns + corpus-time stripping

Chris Grady | @CGFixIT | cgfixit.com

Property	Value
Fallback	Grok via xAI API (user-gated, config-gated, env-gated)
MCP Server	Retrieval-only, sampling = null, no LLM access
Binding	127.0.0.1:8787 (offline-first, CORS-restricted)
Audit	SHA-256 hashed, PII-redacted, append-only JSONL
Browser UI	terminal.html (Soul Console) + extractor.html

Contents

A. System Overview

PsyClaw is a six-layer architecture where each layer has a single responsibility and explicit boundaries. The LangGraph controller is the security enforcement mechanism — routing decisions are made by graph topology, not by prompt instructions. Version 1.3.0 preserves the v1.2.0 CORS middleware and StaticFiles mount and adds a per-IP rate limiter at the top of the request pipeline (before sanitization).

Client Interfaces

Interface	Protocol	Entry Point
CLI / Scripts	HTTP POST	/query, /health, /soul/*
Browser UI (terminal.html)	HTTP	/ (StaticFiles) → /query, /soul/*
Extractor (extractor.html)	HTTP	Corpus extraction tool
LM Studio / Claude Desktop	MCP (JSON-RPC / stdio)	mcp_hybrid_server.py

Layer Stack

Layer	Module(s)	Responsibility
A. Gateway	gate.py, mcp_hybrid_server.py	Rate limit, sanitization, CORS, static serving, GraphState init
B. Controller	graph.py	7-node LangGraph: retrieve → route → respond → audit
C. Retrieval	retrieval/*.py	Corpus indexing, hybrid search (Chroma + BM25 + RRF)
D. Soul	utils/personality.py, data/personality/	Versioned identity, drift-detected, atomic writes, TTL pruning
E. Models	llm/client.py	LM Studio (local), sentence-transformers, Grok (triple-gated)
F. MCP Server	mcp_hybrid_server.py	Retrieval-only tool; sampling = null

B. Gateway Layer

The FastAPI gateway (`gate.py`) and MCP hybrid server are the only external interfaces. Both bind to `127.0.0.1` — no external network exposure by default. Version 1.3.0 promotes rate limiting from configured-but-stubbed to fully active, and runs the rate-limit check before the prompt filter so abusive clients are dropped before any sanitization or graph work executes.

HTTP Endpoints

Endpoint	Method	Description
`/query`	POST	Main entry — rate-limit, sanitize, build GraphState, invoke graph
`/health`	GET	Liveness + dependency health checks (LM Studio, Grok, embeddings)
`/soul`	GET	Current soul content, version number, source path
`/soul/propose`	POST	Diff preview with SHA hashes — does NOT apply
`/soul/apply`	POST	Applies evolution — requires human reason string
`/soul/reload`	POST	Reload soul.md from disk after manual edit
`/`	GET	Serves terminal.html (Soul Console) via StaticFiles mount

Rate Limiter (NEW in v1.3.0)

Implemented as a thread-safe in-memory dict keyed by client IP, with a 60-second sliding window. Each request appends `time.time()` to the per-IP list, prunes timestamps older than the window, and rejects with HTTP 429 if the list reaches `RATE_LIMIT_REQUESTS` (default 60). No external dependency required — slowapi was evaluated and rejected to keep the offline-first dep tree minimal. For distributed deployments the same interface can be swapped to slowapi or Redis without touching the rest of the gateway.

```
_rate_limits = defaultdict(list)
RATE_LIMIT_REQUESTS = 60
RATE_LIMIT_WINDOW = 60 # seconds

def check_rate_limit(client_ip: str) -> bool:
    now = time.time()
    _rate_limits[client_ip] = [
        t for t in _rate_limits[client_ip]
        if now - t < RATE_LIMIT_WINDOW
    ]
    if len(_rate_limits[client_ip]) >= RATE_LIMIT_REQUESTS:
        return False
    _rate_limits[client_ip].append(now)
    return True
```

Gateway Invariants (v1.3.0)

- Rate-limit check runs FIRST (per-IP, 60/min sliding window) — exhausted clients hit 429 before any work
- Banned-pattern prompt filter (13 patterns) runs before any LLM call
- Max input size 4000 chars enforced at gateway level
- CORS restricted to `security.allowed_origins` (config-driven allowlist)
- GraphState initialized with: `query, user_confirmed_online, soul_core`
- All responses follow uniform JSON schema (answer, sources, model_used, retrieval_mode, needs_confirm, error)
- Structured error handling with typed exceptions (RAGError → PromptInjectionError, IndexNotFoundError, LLMServiceError, etc.)
- Telemetry kill-switch env vars set at module top, BEFORE any langchain/chromadb import

CORS Configuration

`CORSMiddleware` reads `security.allowed_origins` from `config.yaml` and restricts cross-origin requests. Default allowed origins: `http://127.0.0.1`, `http://localhost`. Methods limited to GET and POST. Headers restricted to `Content-Type`. Credentials disallowed.

GraphState Schema

Field	Type	Set By
`query`	str	Gateway (from user input)
`user_confirmed_online`	bool | None	Gateway (from client resubmit)
`retrieved_docs`	list[RetrievedDoc]	retrieve node
`top_score`	float	retrieve node
`retrieval_mode`	str	retrieve node (hybrid|vector|bm25|none)
`needs_user_confirm`	bool	route_by_score / user_gate
`answer`	str	local_llm | grok_fallback | offline_best
`answer_model`	str	Responding node (local|grok|offline-best-effort)
`answer_sources`	list[RetrievedDoc]	Responding node
`audit_event`	dict	audit_logger node
`error`	str | None	Any node on failure

C. LangGraph Controller

The controller is a 7-node directed graph where topology IS the security policy. Routing decisions are structural — enforced by graph edges, not by prompt wording. Every execution path terminates at the `audit_logger` node before returning to the gateway. Soul injection happens at the prompt construction level inside `local_llm` and `offline_best` — there is no soul graph node. Evolution is an explicit HTTP endpoint, not a graph operation, per model council consensus: no autonomous self-modification in the graph.

Graph Topology

```
[ENTRY]
   ↓
1. retrieve (Chroma + BM25 + RRF fusion → retrieved_docs, top_score)
   ↓
2. route_score (top_score >= cfg.retrieval.min_score [0.028 RRF]?)
   ├─ score >= threshold ──→ 3. local_llm (soul + query + docs → LM Studio)
   └─ score < threshold ──→ 4. user_gate (sets needs_user_confirm; awaits resubmit)
                                ├─ user=yes + hybrid + grok.enabled ──→ 5.
grok_fallback
                                └─ user=no OR Grok disabled ──→ 6. offline_best
   ↓
7. audit_logger (SHA-256 query hash + PII redact → logs/audit.jsonl) → END
```

All paths converge at `audit_logger`. The convergence is enforced by graph edges, not by code discipline — there is no shortcut path. Personality interactions are also recorded to the shadow DB after audit so soul-level history stays consistent with the audit trail.

D. Retrieval Layer

Retrieval is the unconditional first node in the graph. The hybrid retriever combines semantic (ChromaDB cosine HNSW) and keyword (BM25Okapi) search via Reciprocal Rank Fusion (RRF). If either path fails, the retriever degrades gracefully to whichever path remains viable.

Indexing Pipeline (offline batch)

- `load_corpus()` — recursive read of `.md/.txt` from `data/corpus/`
- `chunk_document(size=512, overlap=50)` with sentence-boundary snapping
- `sanitize_chunk()` — strips banned patterns at ingestion (13 patterns in v1.3)
- `tokenize_and_stem()` — enhanced Porter stemmer (technical-vocab safe)
- `embed_chunks()` — 384-dim via `all-MiniLM-L6-v2` (CPU, batch=32)
- Write Chroma collection → `index/chroma_db/` (cosine HNSW)
- Build BM25 index → `index/bm25.pkl` (pickle: bm25 + chunks + metadata)

Query-Time Hybrid Search

Component	Method	Effective Weight
Semantic	Chroma vector search (embed → cosine sim)	1.0 (equal)
Keyword	BM25 (tokenize + stem → term frequency)	1.0 (equal)
Fusion	RRF (k=60): score = Σ 1/(k + rank_i)	—

Each `SearchResult` carries full RRF provenance: `semantic_score, semantic_rank, keyword_score, keyword_rank, rrf_score, rrf_semantic_contrib, rrf_keyword_contrib`. Fields are exposed in the `SourceInfo` Pydantic response model so clients can inspect why each result ranked where it did.

Design choice (v1.3.0+): PsyClaw uses equal-weighted RRF — each path contributes `1/(k+rank)`, summed. The `vector_weight` / `bm25_weight` keys in `config.yaml` are documentation-only and are not multiplied into the RRF contribution by the current `hybrid_search.py`. Earlier specs mentioned a 0.6/0.4 split as a tunable; in practice equal weighting performs well across the existing corpus and was kept. To opt into true weighted RRF, multiply `contrib *= weight` in both loops AND retune `retrieval.min_score` (halving weights halves max RRF score).

Retrieval Configuration

Parameter	Value	Notes
top_k_semantic	5	Max vector results
top_k_keyword	5	Max BM25 results
rrf_k	60	RRF smoothing constant
max_context_tokens	5000	Token budget for retrieved context
min_score	0.028	RRF fused-rank threshold (NOT cosine sim — different scale)
chunk_size	512	Chars per chunk (sentence-snapped)
chunk_overlap	50	Overlap between adjacent chunks
batch_size	50	Embedding batch size (config); inference batch is 32

E. Soul Governance

Identity ≠ memory ≠ topology. Soul is who the agent IS. Memory is what the agent KNOWS. Topology is what the agent CAN DO. Each is governed independently. v1.3.0 hardens the soul layer in three concrete ways while preserving the original write-order invariant and `threading.Lock` serialization.

v1.3.0 Hardening

Drift Detection. On every `PersonalityManager.__init__`, the SHA-256 of `soul.md` is compared to the latest `sha256` in `soul_versions`. Mismatch (manual edit, tampering, crash corruption) triggers a forensic `audit_log` event AND inserts an automatic recovery version row so the audit trail always reflects true on-disk state.

Atomic Evolution. `apply_evolution` writes to a `soul.md.tmp` sibling file then calls `os.replace()` for a POSIX-atomic rename. Eliminates partial-write corruption risk on crash or power loss during the write.

Automatic TTL Pruning. `maintenance(ttl_days)` is invoked from `__init__` using `personality.interaction_ttl_days` (default 365) so old interaction rows are cleaned on every gateway start. Default raised from 90 → 365 days in v1.3.0 to retain a richer audit window.

Threading Lock Preserved. `threading.Lock` still serializes every SQLite write. Drift recovery, version insert, interaction insert, and TTL prune all hold the lock — concurrent gateway workers cannot corrupt the shadow DB.

Components

- `data/personality/soul.md` — source-of-truth identity file
- `data/personality/psyclaw_soul.db` — SQLite shadow DB (`soul_versions` + `interactions` tables)
- `utils/personality.py` — `PersonalityManager` (lock-serialized SQLite writes)

Write-Order Invariant (Crash-Safe)

```
1. Write proposed soul to soul.md.tmp
2. os.replace(soul.md.tmp, soul.md) # POSIX-atomic rename
3. INSERT new version row into SQLite (under threading.Lock)
4. Update in-memory soul_core (runtime matches persisted state)
5. audit_log({'event': 'soul_evolution_applied', ...})
```

Any crash between steps leaves the system recoverable: `soul.md` is either the old or new content (never partial) and the SQLite version log records the evolution intent.

Soul API Constraints

- `GET /soul` — read-only (current content + version + path)
- `POST /soul/propose` — generates `(current_sha, proposed_sha)` diff WITHOUT applying
- `POST /soul/apply` — requires human-authored `reason` string; no anonymous mutations
- `POST /soul/reload` — re-reads `soul.md` after manual edit
- No endpoint allows silent or automated soul rewrites

Personality Config

Parameter	Value	Description
`personality.enabled`	true	Toggle soul injection
`personality.soul_path`	data/personality/soul.md	File-as-truth
`personality.db_path`	data/personality/psyclaw_soul.db	Shadow DB
`personality.interaction_ttl_days`	365 (was 90 in v1.2)	Prune old interaction logs

F. Model Layer

Model	Type	Endpoint	Gating
all-MiniLM-L6-v2	Embeddings (384d)	CPU local, .emb_cache/	Always available
Qwen 2.5 7B (GGUF)	Chat LLM	127.0.0.1:1234/v1	LM Studio required
grok-beta	Chat LLM (external)	api.x.ai/v1	Triple-gated (see below)

LocalLLMClient

Timeout 720 s (long-context inference), temperature 0.3, max_tokens 5000. Defensive: all client calls raise typed `LLMServiceError` subclasses mapped to HTTP responses by the gateway.

GrokClient — Triple Gate

- `app.mode == 'hybrid'` (config flag)
- `models.grok.enabled == true` (config flag)
- `user_confirmed_online == true` (per-query, set by client resubmit)

All three must be simultaneously true. The Grok client validates each precondition defensively even though the graph topology already prevents invalid calls — defense in depth. `policy.fallback.send_local_context_to_grok = false` by default: retrieved docs are NOT forwarded to Grok unless the user explicitly opts in.

G. MCP Server

Property	Value
Protocol	JSON-RPC over stdio
Server Name	psyclaw-hybrid-rag v1.0.0
Tool	hybrid_search
Input schema	{ query: string, top_k?: int, mode?: 'hybrid'|'vector'|'bm25' }
Sampling	null — cannot invoke any LLM (retrieval-only by protocol design)
Write access	None (read-only against existing indices)

The `sampling = null` constraint is structural, not policy. The MCP server cannot autonomously invoke an LLM. Any LLM processing must go through the gateway's LangGraph controller with full audit coverage — the MCP path bypasses none of the five security invariants.

H. Security and Sanitization

v1.3.0 expands the prompt-injection sanitizer from 8 to 13 OWASP-aligned banned patterns. The sanitizer operates at two levels: user input (reject on match, raises `PromptInjectionError`) and corpus chunks at ingestion (strip + replace with `[FILTERED]`). Patterns are config-driven and hot-reloadable — adding a pattern is a `config.yaml` change, not a code change.

Banned Patterns (13, OWASP-aligned)

Pattern	OWASP Category	Status
`ignore previous instructions`	LLM01 — Prompt Injection	v1.2 (8 base)
`ignore all instructions`	LLM01 — Prompt Injection	v1.2 (8 base)
`you are now in developer mode`	LLM01 — Prompt Injection	v1.2 (8 base)
`as an ai with no restrictions`	LLM01 — Jailbreak	v1.2 (8 base)
`system prompt:`	LLM01 — Prompt Extraction	v1.2 (8 base)
`disregard your instructions`	LLM01 — Prompt Injection	v1.2 (8 base)
`pretend you are`	LLM01 — Role Hijacking	v1.2 (8 base)
`jailbreak`	LLM01 — Jailbreak	v1.2 (8 base)
`do anything now`	LLM01 — Prompt Injection	NEW in v1.3
`act as uncensored`	LLM01 — Jailbreak	NEW in v1.3
`bypass safety`	LLM01 — Prompt Injection	NEW in v1.3
`DAN`	LLM01 — Role Hijacking	NEW in v1.3
`ignore safety`	LLM01 — Prompt Injection	NEW in v1.3

PII Redaction (audit logger)

Type	Pattern	Replacement
`Emails`	Standard email regex	[REDACTED_EMAIL]
`IPv4`	Dot-notation	[REDACTED_IP]
`AWS Keys`	AKIA[0-9A-Z]{16}	[REDACTED_SECRET]
`Slack Tokens`	xoxb-[0-9a-zA-Z-]+	[REDACTED_SECRET]
`API Keys`	sk-[a-zA-Z0-9]{20,}	[REDACTED_SECRET]

Rate Limiting

`security.rate_limit.enabled = true`, `requests_per_minute = 60`. Enforced before sanitization in `gate.py`. Returns HTTP 429 with structured error JSON `{error: 'Rate limit exceeded (60/min)', code: 'RATE_LIMIT'}` and emits an `audit_log` event with the offending client IP for forensic review.

I. Browser UI

Both UIs are served via the StaticFiles mount on the gateway and run fully against `http://127.0.0.1:8787` — no external JS dependencies, no CDN-loaded scripts.

- terminal.html (~29 KB) — primary Soul Console: query input → `/query`, real-time answer + sources + retrieval metadata, Soul panel for load/propose/apply/reload
- extractor.html (~18 KB) — corpus extraction utility (carried over unchanged from v1.2.0)

J. Test Suite

Original 8 pytest files + PowerShell smoke test remain. v1.3.0 adds three new test files for the v1.3 features. All tests use mocked external deps — no live LM Studio / Chroma / Grok required.

File	Coverage
`tests/conftest.py`	Shared fixtures + TEST_CONFIG mirroring prod
`tests/test_graph.py`	All 7 LangGraph paths: local, grok, offline, gate, error
`tests/test_gate.py`	/query, /health, /soul/*, error responses
`tests/test_personality.py`	PersonalityManager: init, propose, apply, reload
`tests/test_sanitizer.py`	Banned patterns, max length, corpus sanitization
`tests/test_audit.py`	SHA-256 hashing, PII redaction, JSONL output
`tests/test_hybrid_search.py`	Semantic, keyword, fusion, graceful degradation
`tests/test_stemmer.py`	Plurals, verb forms, technical terms
`tests/apipsTest.ps1`	PowerShell API smoke tests
`tests/test_rate_limit.py (NEW v1.3)`	Sliding window, expiry, per-IP isolation
`tests/test_personality_changes.py (NEW v1.3)`	Drift detection, atomic apply, TTL prune, version ordering
`tests/test_endpoints_mocked.py (NEW v1.3)`	Full /query, /soul/*, rate-limit, health under heavy mocks

Pre-flight test runner:

```
python -m pytest test_personality_changes.py test_rate_limit.py -q
python -m pytest tests/ -q # full suite under mocks
```

K. Deployment & Validation

v1.3.0 is a drop-in upgrade for v1.2.0 deployments. No database migration required — the existing `psyclaw_soul.db` schema is compatible. The drift-detection on init will produce one `auto-recovery` version row on first startup if the on-disk soul has diverged from the last DB-recorded SHA, which is expected and forensically logged.

Recommended Upgrade Path

- Replace `gate.py`
- Replace `utils/personality.py`
- Replace `utils/sanitizer.py`
- Replace `config.yaml` (or merge: rate_limit, banned_patterns, interaction_ttl_days)
- Run pre-flight: `python -m pytest test_personality_changes.py test_rate_limit.py -q`
- Start gateway: `uvicorn gate:app --host 127.0.0.1 --port 8787`
- Smoke test: `curl -X POST http://127.0.0.1:8787/query -d '{"query":"hi"}' -H 'Content-Type: application/json'`
- Verify: `curl http://127.0.0.1:8787/health` shows all deps healthy
- Confirm: rate-limit and drift events appear in `logs/audit.jsonl`

Rollback Plan

If v1.3.0 misbehaves: stop the gateway, restore the four replaced files from the v1.2.0 backup, restart. The `psyclaw_soul.db` schema is forward-compatible with v1.2.0 — no data loss. Any v1.3 auto-recovery rows in `soul_versions` are inert under v1.2.0 (just additional history).

L. v1.3.0 Change Log

Concrete code-level diff against v1.2.0. All changes preserve topology = enforcement and identity ≠ memory ≠ topology. No graph topology changes; all hardening lives in gate, soul, and sanitizer.

Gateway (`gate.py`)

Added in-memory rate limiter (sliding 60 req/min per-IP) BEFORE sanitization. Version bumped to 1.3.0. Telemetry kill-switch env vars verified at import with status print. `FastAPI Request` object now passed into `/query` for client-IP extraction.

Soul Governance (`utils/personality.py`)

SHA-256 drift detection on load with forensic audit + automatic recovery version row. Atomic `apply_evolution` via `.tmp + tmp_path.replace()`. TTL prune now runs automatically in `__init__`. `threading.Lock` preserved across all SQLite writes.

Security & Sanitization (`utils/sanitizer.py` + `config.yaml`)

Banned patterns expanded to 13 OWASP-aligned (added 'do anything now', 'act as uncensored', 'bypass safety', ' DAN ', 'ignore safety'). Config-driven and hot-reloadable. Corpus-time sanitizer also uses the expanded list with `[FILTERED]` replacement.

Testing

New: `test_rate_limit.py`, `test_personality_changes.py` (drift / atomic / TTL), `test_endpoints_mocked.py`. All original 8 tests still pass under mocks.

Config

`security.rate_limit.enabled = true`; 5 new injection patterns added; `personality.interaction_ttl_days` default raised from 90 → 365.

Documentation

This v1.3.0 PDF; `psyclaw_suggestions_fix.md` with full refactor notes and validation checklist (note: that file's `slowapi` recommendation was evaluated and rejected — the in-memory dict approach is canonical).

Note: Rate limiting implemented as a lightweight in-memory dict (no external `slowapi` dep). For distributed deployments, the same interface can be swapped for slowapi or Redis without touching the rest of the gateway.

M. Security Invariants

The five topology-enforced guarantees remain exactly as in v1.2.0. v1.3.0 adds runtime verification and hardened code paths (rate limiter, drift detection, atomic writes, expanded sanitizer) that make the invariants even harder to violate.

#	Invariant	v1.3.0 Enforcement
1	RAG-First	retrieve node is still the unconditional ENTRY → retrieve edge. No prompt can add a bypass edge.
2	Topology = Policy	route_score is a conditional edge, not an LLM decision. Injected prompts cannot mutate graph topology.
3	Triple-Gated External Access	grok_fallback only reachable via app.mode=='hybrid' AND models.grok.enabled AND user_confirmed_online.
4	Audit Convergence	ALL response nodes → audit_logger → END. Personality interactions also recorded to shadow DB after audit.
5	Soul Governance	apply requires reason; propose is diff-only; drift auto-recovers with forensic log; atomic writes prevent partial-state corruption.

BOTTOM LINE

- Three invariants: retrieval first, topology = policy, identity != memory != topology.
- v1.3.0 closes the rate-limit gap, expands the sanitizer to 13 patterns, and hardens the soul layer with drift detection + atomic writes.
- No graph topology changes. No new external dependencies. No database migration.
- PsyClaw remains fully offline-first, RAG-enforced, and identity-governed.

— Chris Grady (@CGFixIT) | May 2026 | github.com/CGFixIT/PsyClaw | cgfixit.com/zSafeClaw